

The **StringTokenizer** class in Java belongs to the `java.util` package and is used to break a string into smaller portions called **tokens** based on specified delimiters.

`StringTokenizer` is a useful to divide the string into smaller **tokens** based on delimiters.

The current Java API documents **three constructors** and six main tokenizer methods; it also implements `Enumeration<Object>`. ([Oracle Documentation](#))

Official Java documentation: [StringTokenizer – Java API Documentation](#)

1. What is StringTokenizer?

`StringTokenizer` belongs to the `java.util` package and is used to break a string into smaller parts called **tokens**.

For example:

```
"Java is an Object Oriented Language"
```

Using space as the delimiter:

```
Java
is
an
Object
Oriented
Language
```

A **delimiter** is a character that separates tokens. The default delimiters are space, tab, newline, carriage return and form feed. ([Oracle Documentation](#))

2. Three Constructors of StringTokenizer

Constructor	Purpose
<code>StringTokenizer(String str)</code>	Uses default delimiters
<code>StringTokenizer(String str, String delim)</code>	Uses specified delimiters
<code>StringTokenizer(String str, String delim, boolean returnDelims)</code>	Specifies delimiters and whether delimiters should also be returned

These are the three constructors provided by the Java API. ([Oracle Documentation](#))

3. Complete Java Program – Constructors and Methods

```
package com.company.string;

import java.util.StringTokenizer;

public class StringTokenizerDemo {

    public static void main(String[] args) {

        //
        =====
        // 1. Constructor: StringTokenizer(String str)
        //
        =====
        System.out.println("===== CONSTRUCTOR 1
        =====");

        String str1 = "Java is an Object Oriented Language";

        // Default delimiters are:
        // space, tab, newline, carriage return and form feed
        StringTokenizer st1 = new StringTokenizer(str1);

        System.out.println("Original String: " + str1);

        while (st1.hasMoreTokens()) {

            // nextToken() returns the next available token
            String token = st1.nextToken();

            System.out.println(token);
        }

        //
        =====
        // 2. Constructor: StringTokenizer(String str, String
        delim)
        //
        =====
        System.out.println("\n===== CONSTRUCTOR 2
        =====");

        String str2 = "Java,Python,C++,JavaScript";

        // Comma is specified as the delimiter
```

```

StringTokenizer st2 =
    new StringTokenizer(str2, ",");

System.out.println("Original String: " + str2);

while (st2.hasMoreTokens()) {

    System.out.println(st2.nextToken());
}

//
=====
// 3. Constructor:
// StringTokenizer(String str, String delim,
//                  boolean returnDelims)
//
=====
System.out.println("\n===== CONSTRUCTOR 3
=====");

String str3 = "Java,Python,C++";

// true means delimiter will also be returned as a token
StringTokenizer st3 =
    new StringTokenizer(str3, ",", true);

System.out.println("Original String: " + str3);

while (st3.hasMoreTokens()) {

    System.out.println(st3.nextToken());
}

//
=====
// 4. countTokens()
//
=====
System.out.println("\n===== countTokens()
=====");

String str4 = "Java is easy to learn";

StringTokenizer st4 =
    new StringTokenizer(str4);

```

```

        System.out.println("Number of tokens: "
            + st4.countTokens());

        //
=====
        // 5. hasMoreTokens()
        //
=====
        System.out.println("\n===== hasMoreTokens()
=====");

        String str5 = "Java Python C++";

        StringTokenizer st5 =
            new StringTokenizer(str5);

        while (st5.hasMoreTokens()) {

            System.out.println("Token available: "
                + st5.nextToken());

        }

        //
=====
        // 6. nextToken()
        //
=====
        System.out.println("\n===== nextToken()
=====");

        String str6 = "Welcome to Java";

        StringTokenizer st6 =
            new StringTokenizer(str6);

        System.out.println(st6.nextToken());
        System.out.println(st6.nextToken());
        System.out.println(st6.nextToken());

        //
=====
        // 7. nextToken(String delim)

```

```

//
=====
System.out.println("\n===== nextToken(String delim)
=====");

String str7 = "Java,Python,C++";

// Initially comma is the delimiter
StringTokenizer st7 =
    new StringTokenizer(str7, ",");

System.out.println(st7.nextToken());

// Change delimiter for subsequent tokenization
System.out.println(st7.nextToken());

//
=====
// 8. hasMoreElements()
//
=====
System.out.println("\n===== hasMoreElements()
=====");

String str8 = "One Two Three";

StringTokenizer st8 =
    new StringTokenizer(str8);

while (st8.hasMoreElements()) {

    System.out.println(st8.nextElement());
}

//
=====
// 9. nextElement()
//
=====
System.out.println("\n===== nextElement()
=====");

String str9 = "Apple Mango Orange";

StringTokenizer st9 =

```

```
        new StringTokenizer(str9);

    while (st9.hasMoreElements()) {

        Object element = st9.nextElement();

        System.out.println(element);

    }

}
```

4. Important Methods Explained

The Java API lists the following methods for StringTokenizer: `countTokens()`, `hasMoreElements()`, `hasMoreTokens()`, `nextElement()`, `nextToken()`, and `nextToken(String delim)`. ([Oracle Documentation](#))

1. `hasMoreTokens()`

Checks whether another token is available.

```
StringTokenizer st =
    new StringTokenizer("Java Python C++");

while (st.hasMoreTokens()) {
    System.out.println(st.nextToken());
}
```

Output:

```
Java
Python
C++
```

It returns:

```
true
```

if another token is available, otherwise:

```
false
```

2. `nextToken()`

Returns the next token.

```
StringTokenizer st =  
    new StringTokenizer("Java Python C++");  
  
System.out.println(st.nextToken());  
System.out.println(st.nextToken());  
System.out.println(st.nextToken());
```

Output:

```
Java  
Python  
C++
```

If there are no more tokens, calling `nextToken()` results in `NoSuchElementException`.
([C# Corner](#))

3. `countTokens()`

Returns the number of tokens that can still be obtained using `nextToken()`.

```
StringTokenizer st =  
    new StringTokenizer("Java is easy");  
  
System.out.println(st.countTokens());
```

Output:

```
3
```

Important:

```
System.out.println(st.countTokens());  
  
System.out.println(st.nextToken());  
  
System.out.println(st.countTokens());
```

Output:

```
3  
Java  
2
```

So, `countTokens()` counts the **remaining tokens**, not necessarily the original number of tokens. ([Oracle Documentation](#))

4. `hasMoreElements()`

This method works essentially the same way as `hasMoreTokens()`.

```
StringTokenizer st =  
    new StringTokenizer("Java Python");  
  
while (st.hasMoreElements()) {  
    System.out.println(st.nextElement());  
}
```

`hasMoreElements()` returns the same value as `hasMoreTokens()`. ([Oracle Documentation](#))

5. `nextElement()`

Returns the next token as an `Object`.

```
StringTokenizer st =  
    new StringTokenizer("Java Python");  
  
Object obj = st.nextElement();  
  
System.out.println(obj);
```

Output:

Java

The difference is mainly the return type:

```
nextToken()    → String  
nextElement() → Object
```

The Java API specifies that `nextElement()` returns the same value as `nextToken()`, but its declared return type is `Object`. ([Oracle Documentation](#))

6. `nextToken(String delim)`

This method allows a **new delimiter** to be specified when obtaining the next token.

```
StringTokenizer st =  
    new StringTokenizer("Java,Python;C++", " ,");  
  
System.out.println(st.nextToken());  
  
System.out.println(st.nextToken(";"));
```

It is useful when the delimiter needs to be changed during tokenization. ([Oracle Documentation](#))

5. Understanding `returnDelims`

This is particularly important for students.

false

```
StringTokenizer st =  
    new StringTokenizer("Java,Python,C++", " ,", false);
```

Output:

```
Java  
Python  
C++
```

The comma is only used as a separator.

true

```
StringTokenizer st =  
    new StringTokenizer("Java,Python,C++", " ,", true);
```

Output:

```
Java  
,  
Python  
,  
C++
```

When `returnDelims` is `true`, delimiter characters are also returned as tokens. ([Oracle Documentation](#))

6. A Better Classroom Example

For teaching students, I recommend this simple example first:

```
package com.company.string;

import java.util.StringTokenizer;

public class StringTokenizerExample {

    public static void main(String[] args) {

        String sentence =
            "Java is a powerful programming language";

        // Create StringTokenizer
        StringTokenizer tokenizer =
            new StringTokenizer(sentence);

        System.out.println("Original String:");
        System.out.println(sentence);

        System.out.println("\nTokens:");

        // Check whether token is available
        while (tokenizer.hasMoreTokens()) {

            // Get next token
            String token = tokenizer.nextToken();

            System.out.println(token);

        }
    }
}
```

Output:

```
Original String:
Java is a powerful programming language
```

```
Tokens:
Java
```

is
a
powerful
programming
language

Concept students should understand

String
↓
StringTokenizer
↓
Delimiter
↓
Tokens

For example:

"Java,Python,C++"
↓
comma
↓
Java | Python | C++

7. One Program to Demonstrate All Six Methods

For your **BCSC1002 OOP / Java class**, this compact program is probably the best classroom demonstration:

```
package com.company.string;

import java.util.StringTokenizer;

public class StringTokenizerMethods {

    public static void main(String[] args) {

        String data = "Java,Python,C++,JavaScript";

        StringTokenizer st =
            new StringTokenizer(data, ",");

        // 1. countTokens()
        System.out.println("Number of Tokens: "
            + st.countTokens());
```

```

// 2. hasMoreTokens()
while (st.hasMoreTokens()) {

    // 3. nextToken()
    String token = st.nextToken();

    System.out.println("Token: " + token);
}

// Create tokenizer again because
// previous tokenizer has consumed all tokens
st = new StringTokenizer(data, ",");

System.out.println("\nUsing Enumeration Methods:");

// 4. hasMoreElements()
while (st.hasMoreElements()) {

    // 5. nextElement()
    Object obj = st.nextElement();

    System.out.println("Element: " + obj);
}

// 6. nextToken(String delimiter)
System.out.println("\nChanging Delimiter:");

String data2 = "Java-Python-C++";

StringTokenizer st2 =
    new StringTokenizer(data2, "-");

System.out.println(st2.nextToken());
System.out.println(st2.nextToken());
System.out.println(st2.nextToken());
}
}

```

Quick revision table for students

Constructor / Method	Purpose
<code>StringTokenizer(str)</code>	Tokenize using default delimiters
<code>StringTokenizer(str, delim)</code>	Tokenize using specified delimiter
<code>StringTokenizer(str, delim,</code>	Control whether delimiters are returned

Constructor / Method	Purpose
true/false)	
hasMoreTokens()	Check whether token is available
nextToken()	Get next token
nextToken(delim)	Get next token using a specified delimiter
countTokens()	Count remaining tokens
hasMoreElements()	Check whether another element exists
nextElement()	Get next token as Object

StringTokenizer is a legacy class; modern Java code often uses `String.split()` or other APIs for ordinary string splitting. The Java documentation itself describes StringTokenizer as a legacy compatibility class. ([Oracle Documentation](#))